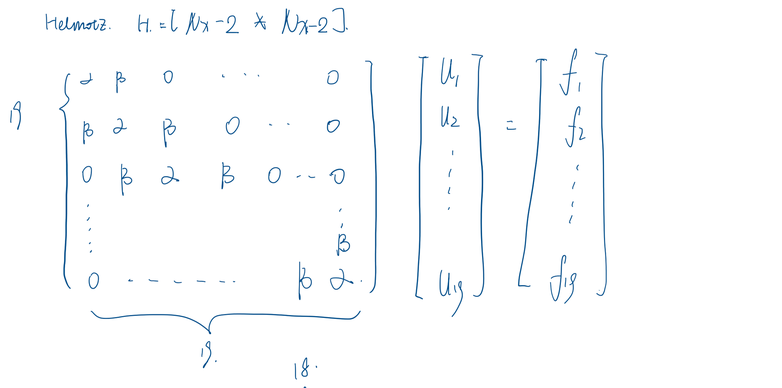
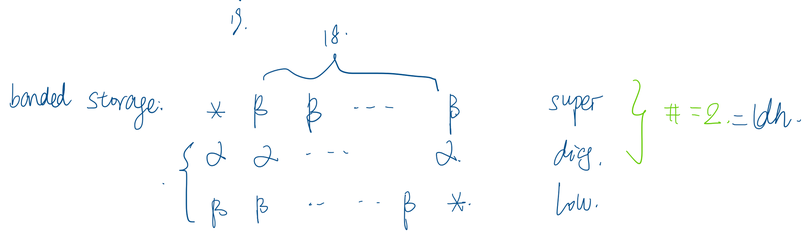
Ex 4-5 using banded derivation

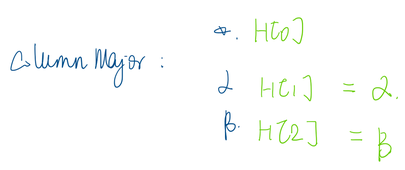
1/30/24 08:07

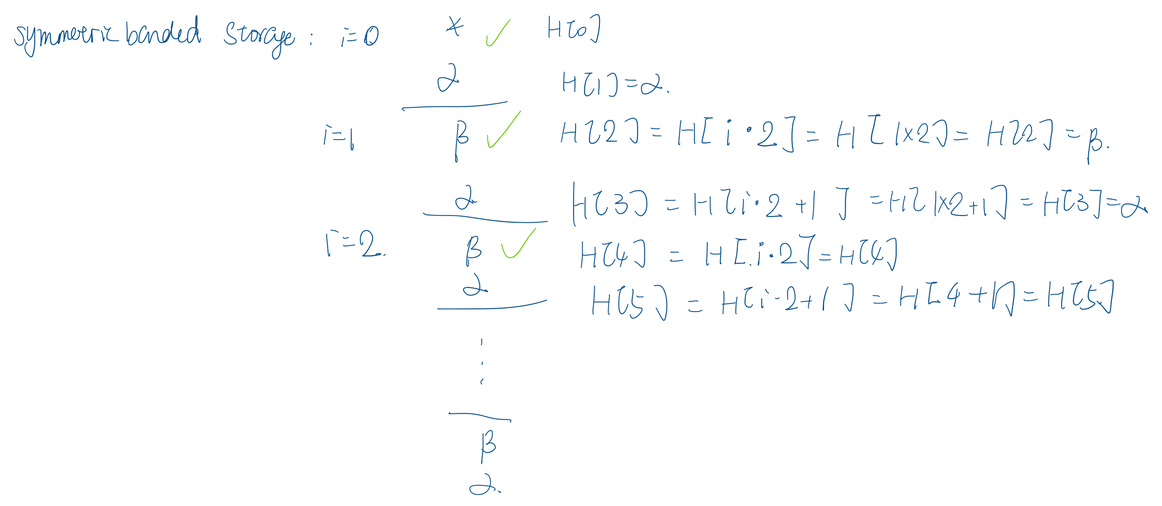
Exercise 4.5

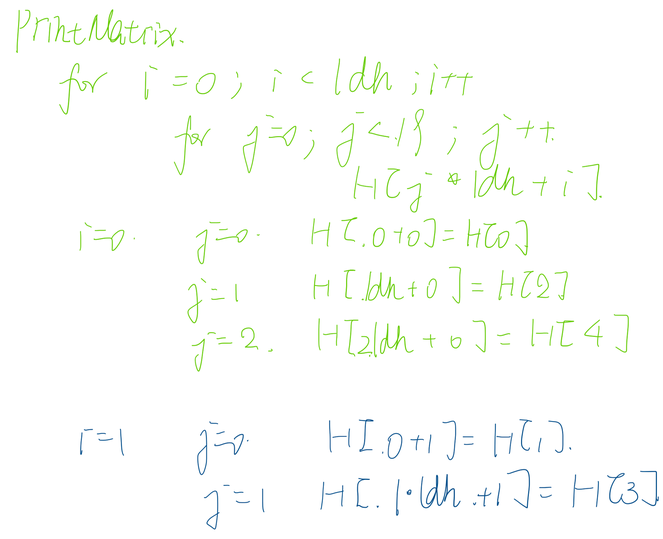
Further optimise the memory and computational efficiency of your solution to Exercise 4.4 by using banded storage and calling appropriate BLAS routines.

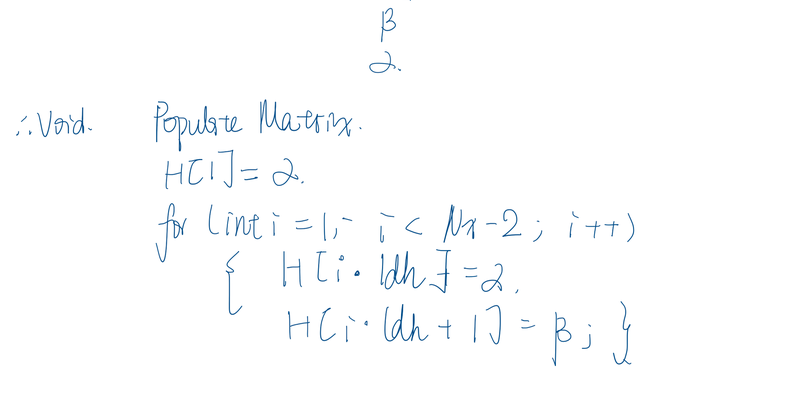


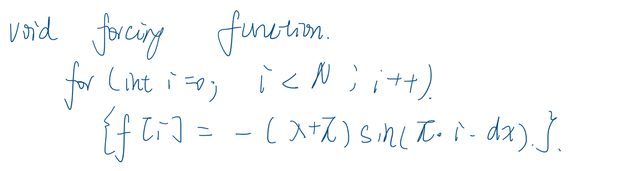


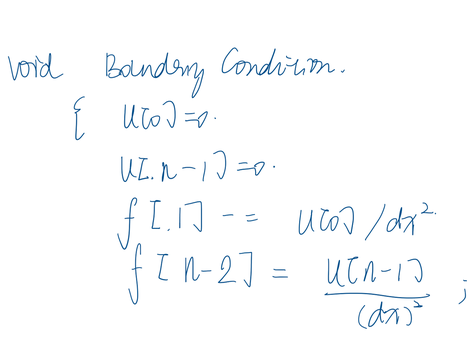


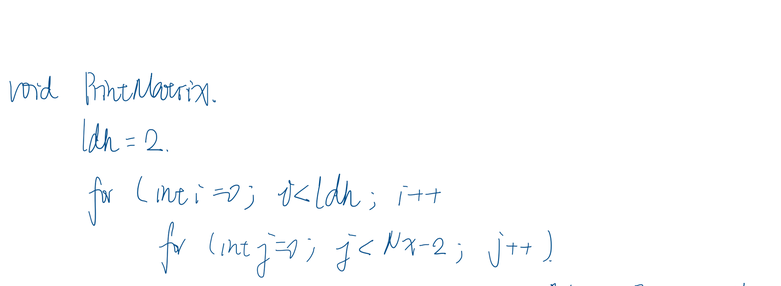













```
/**
 * @file cg_helmholtz_sb.cpp
 *
 * High-performance Computing
 *
 * Solution to Exercise 4.5.
 *
 * Solves Helmholtz equation using Conjugate Gradient algorithm.
 *
 * The Helmholtz matrix is tri-diagonal and symmetric. We adapt the conjugate
 * gradient algorithm to instead use a symmetric banded storage, thereby
 * significantly enhancing the efficiency.
 */
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
#include "cblas.h"
/**
 * @brief Populate Helmholtz symmetric banded matrix (upper only)
 *
 * @param   nsv     Leading dimension of matrix
 * @param   H       Pointer to matrix storage of size 2*nsv
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void FillHelmholtzMatrix(int nsv, double* H, double lam, double dx) {
    const int ldh = 2;      // Diagonal and upper diagonal
    const double oodx2 = 1.0/dx/dx;
    H[1] = -lam - 2.0*oodx2;
    for (int i = 1; i < nsv; ++i) {
        H[i*ldh    ] = oodx2;
        H[i*ldh + 1] = -lam - 2.0*oodx2;
    }
}
/**
 * @brief Fills the forcing vector with
 *        \f$ f = -(\lambda + \pi^2) \sin(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void FillForcingFunction1(int n, double* f, double lam, double dx) {
    for (int i = 0; i < n; ++i) {
        f[i] = -(lam + M_PI*M_PI)*sin(M_PI*i*dx);
    }
}
/**
 * @brief Fills the forcing vector with
 *        \f$ f = -(\lambda + \pi^2) \cos(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void FillForcingFunction2(int n, double* f, double lam, double dx) {
    for (int i = 0; i < n; ++i) {
        f[i] = -(lam + M_PI*M_PI)*cos(M_PI*i*dx);
    }
}
/**
 * @brief Enforces zero Dirichlet boundary conditions.
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to forcing term storage of length n
 * @param   u       Pointer to solution vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void EnforceBoundaryConditions1(int n, double* f, double* u, double lam, double dx) {
    u[0] = sin(0);
    u[n-1] = sin(M_PI*(n-1)*dx);
    f[1] -= u[0]/dx/dx;
    f[n-2] -= u[n-1]/dx/dx;
}
/**
 * @brief Enforces zero Dirichlet boundary conditions.
 *
 * @param   n       Vector dimension
 * @param   f       Pointer to forcing term storage of length n
 * @param   u       Pointer to solution vector storage of length n
 * @param   lam     Lambda coefficient
 * @param   dx      Grid spacing
 */
void EnforceBoundaryConditions2(int n, double* f, double* u, double lam, double dx) {
    u[0] = cos(0);
    u[n-1] = cos(M_PI*(n-1)*dx);
    f[1] -= u[0]/dx/dx;
    f[n-2] -= u[n-1]/dx/dx;
}
/**
 * @brief Computes the exact solution for first problem with
 *        \f$ u = \sin(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   e       Pointer to solution vector storage of length n
 * @param   dx      Grid spacing
 */
void ExactSolution1(int n, double* e, double dx) {
    for (int i = 0; i < n; ++i) {
        e[i] = sin(M_PI*i*dx);
    }
}
/**
 * @brief Computes the exact solution for first problem with
 *        \f$ u = \cos(\pi x) \f$
 *
 * @param   n       Vector dimension
 * @param   e       Pointer to solution vector storage of length n
 * @param   dx      Grid spacing
 */
void ExactSolution2(int n, double* e, double dx) {
    for (int i = 0; i < n; ++i) {
        e[i] = cos(M_PI*i*dx);
    }
}
/**
 * @brief Prints a symmetric tri-banded matrix in column-major format.
 * Note that for a symmetric matrix, only the upper diagonals or lower
 * diagonals need to be stored.
 *
 * @param   nsv     Matrix dimension
 * @param   H       Pointer to matrix storage of size 2*nsv
 */
void PrintMatrix(int nsv, double* H) {
    const int ldh = 2;
    cout.precision(4);
    for (int i = 0; i < ldh; ++i) {
        for (int j = 0; j < nsv; ++j) {
            cout << setw(6) << H[j*ldh+i] << " ";
        }
        cout << endl;
    }
    cout << endl;
}
/**
 * @brief Prints a vector
 *
 * @param   n       Vector dimension
 * @param   u       Pointer to vector storage of length n
 * @param   dx      Grid spacing
 */
void PrintVector(int n, double* u, double dx) {
    for (int i = 0; i < n; ++i) {
        cout << i*dx << "  " << u[i] << endl;
    }
    cout << endl;
}

/**
 * @brief Solve the matrix problem \f$ Hu=f \f$ using the conjugate gradient
 * algorithm.
 *
 * @param   nsv     Dimension of matrix system
 * @param   H       Pointer to matrix storage of size ldh*nsv containing
 *                  the symmetric banded matrix.
 * @param   f       Pointer to vector of length nsv containing forcing for the
 *                  unknown degrees of freedom.
 * @param   u       Pointer to solution vector of length nsv, for the unknown
 *                  degrees of freedom.
 */
void SolveConjugateGradient(int nsv, double* H, double* f, double* u) {
    int   ldh = 2;
    double* r = new double[nsv];
    double* p = new double[nsv];
    double* t = new double[nsv]; //temp
    int k;
    double alpha;
    double beta;
    double eps;
    double tol = 1e-08;
    cblas_dcopy(nsv, f, 1, r, 1);     // r_0 = b (i.e. f)
    cblas_dsbmv(CblasColMajor, CblasUpper, nsv, 1, -1.0, H, ldh,
                u, 1, 1.0, r, 1); // r_0 = b - A x_0
    cblas_dcopy(nsv, r, 1, p, 1);     // p_0 = r_0
    k = 0;
    do {
        cout << "Iteration " << k << endl;
        cblas_dsbmv(CblasColMajor, CblasUpper, nsv, 1, 1.0, H, ldh,
                    p, 1, 0.0, t, 1);
        alpha = cblas_ddot(nsv, t, 1, p, 1);
        alpha = cblas_ddot(nsv, r, 1, r, 1) / alpha; // compute alpha_k
        beta  = cblas_ddot(nsv, r, 1, r, 1);
        cblas_daxpy(nsv, alpha, p, 1, u, 1);  // x_{k+1} = x_k + alpha_k p_k
        cblas_daxpy(nsv, -alpha, t, 1, r, 1); // r_{k+1} = r_k - alpha_k A p_k
        eps = cblas_dnrm2(nsv, r, 1);
        cout << "eps: " << eps << " tol=" << tol << endl;
        if (eps < tol) {
            break;
        }
        beta = cblas_ddot(nsv, r, 1, r, 1) / beta;
        cblas_dcopy(nsv, r, 1, t, 1);
        cblas_daxpy(nsv, beta, p, 1, t, 1);
        cblas_dcopy(nsv, t, 1, p, 1);
        k++;
    } while (k < 500);
    delete[] r;
    delete[] p;
    delete[] t;
}
/**
 * @brief Solves the Helmholtz problem for two different forcing terms.
 */
int main() {
    const int    n   = 21;          // Number of grid-points
    const int    nsv = n - 2;       // Number of unknown DOFs
    const int    ldh = 2;           // Leading dimension of H (D + U)
    const double lam = 1.0;         // Value of Lambda
    const double L   = 1.0;         // Length of domain
    const double dx  = L / (n - 1); // Grid-point spacing
    double* H = new double[ldh*nsv];// Helmholtz matrix storage
    double* u = new double[n];      // Solution vector
    double* f = new double[n];      // Forcing vector
    double* e = new double[n];      // Exact solution vector
    // Generate the Helmholtz matrix in symmetric banded storage.
    FillHelmholtzMatrix(nsv, H, lam, dx);
    cout << "Helmholtz matrix (symmetric banded storage): " << endl;
    PrintMatrix(nsv, H);
    // Problem 1 with f = -(lambda + pi^2) sin(pi x)
    FillForcingFunction1(n, f, lam, dx);
    cblas_dscal(n, 1.0, u, 1);      // x_0 = 0
    EnforceBoundaryConditions1(n, f, u, lam, dx);
    SolveConjugateGradient(nsv, H, f+1, u+1);
    cout << "Solution: " << endl;
    PrintVector(n, u, dx);
    cout << "Exact: " << endl;
    ExactSolution1(n, e, dx);
    PrintVector(n, e, dx);
    cblas_daxpy(n, -1.0, u, 1, e, 1);
    double err1 = cblas_dnrm2(n, e, 1);
    // Problem 2 with f = -(lambda + pi^2) cos(pi x)
    FillForcingFunction2(n, f, lam, dx);
    cblas_dscal(n, 0.0, u, 1);      // x_0 = 0
    EnforceBoundaryConditions2(n, f, u, lam, dx);
    SolveConjugateGradient(nsv, H, f+1, u+1);
    cout << "Solution: " << endl;
    PrintVector(n, u, dx);
    cout << "Exact: " << endl;
    ExactSolution2(n, e, dx);
    PrintVector(n, e, dx);
    cblas_daxpy(n, -1.0, u, 1, e, 1);
    double err2 = cblas_dnrm2(n, e, 1);
    // Print the resulting errors
    cout << "Error (problem 1): " << err1 << endl;
    cout << "Error (problem 2): " << err2 << endl;
    // Clean up
    delete[] H;
    delete[] u;
    delete[] f;
    delete[] e;
}
```